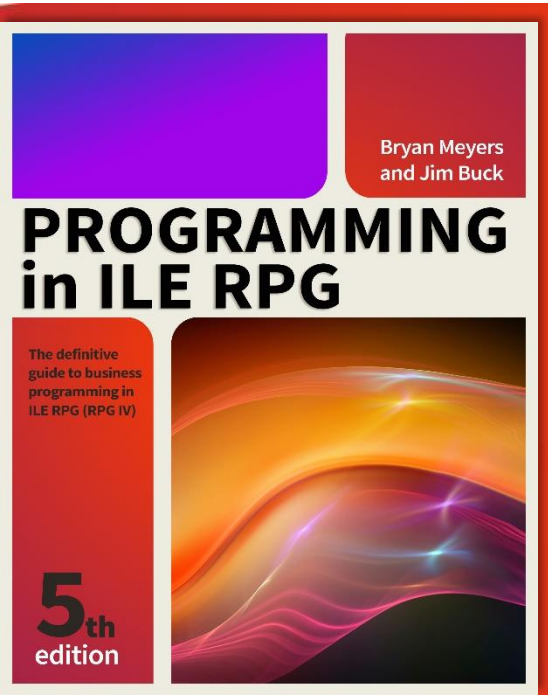
Chapter 6:
Using Arithmetic Operations and Functions



Presentation © Copyright 2015, Enskill.com

Simple Numeric Assignment

- Eval (Evaluate Expression) operation assigns value to variable
  - Target variable and expression must be compatible
    - Eval will not directly convert data types
- Explicitly coding Eval not usually necessary in free format

```
Eval Counter = 0;
Eval Taxrate = .045;
Eval Absolutezero = -273.16;
Eval Amtowed = Balancedue;
```

```
Counter = 0;
Taxrate = .045;
Absolutezero = -273.16;
Amtowed = Balancedue;
```

Assigning Values with Figurative Constants

- *Zero (or *Zeros) fills numeric variables with zeros
- *Hival is maximum positive value
  - All 9s and plus sign (+) for packed or zoned numeric variable
- *Loval is minimum possible value
  - All 9s and minus sign (-) for packed or zoned numeric variable
- Other numeric representations have different values for *Hival and *Loval

```
Totalsales = *Zeros;
Count = *Loval;
```

Using Eval for Arithmetic

- Arithmetic operators
  - + (Addition)
  - - (Subtraction)
  - * (Multiplication)
  - / (Division)
  - ** (Exponentiation)
- Expressions use normal order of operator precedence
  - Parentheses may alter order or clarify expression

Mathematical Order of Precedence

Working from left to right:
First: Exponentiation **
Second: Multiply * Division /
Third: Addition + Subtraction –

Examples:
10 + 2 * 7 evaluates to 24
5 + 30 / 2 – 4 * 3 evaluates to 8

```
Pay = Hourlyrate * 40 +
      1.5 * Hourlyrate * (Hoursworked - 40);
```

Using Eval for Arithmetic

- Compound operators enable concise coding of some expressions
  - +=
  - -=
  - *=
  - /=
  - **=

```
Counter += 1;                    // Same as Counter = Counter + 1
Grandtotal += Empsalary;         // Same as Grandtotal = Grandtotal + Empsalary
Countdown -= 1;                  // Same as Countdown = Countdown – 1
Inventory -= Orderqty;           // Same as Inventory = Inventory - Orderqty

Totalsale += Price + Price * Taxrate;
              // Same as Totalsale = Totalsale + (Price + Price * Taxrate);
```

Numeric Overflow, Truncation

- Value too large for result
- If value has too many high order digits for result, overflow runtime error occurs
- If value has too many decimals for result, truncation occurs
  - Not an error, but caution should be taken

Resulting Sizing Suggestions

Precision --- how many digits
Scale --- how many of the digits are to the right of the decimal point

- Addition, Subtraction
  - Result precision should be at least one more digit than largest operand
  - Result scale should match operand with most decimals
- Multiplication
  - Add multiplier precision values
  - Add multiplier scale values
- Division
  - When divisor is 1 or more, dividend and result precision should match
  - When divisor is less than 1, test representative values
  - Result scale should meet program scale requirements

Rounding

"half-adjust" is an accounting term used to mean "to round"

- Half-adjusting operation extender (H) rounds expression value to proper result scale
  - Extender requires explicit coding of Eval
- Rounding occurs as last step of evaluating expression

```
Eval(h) Interest = Rate * Loanamt;
Eval(h) Avgamount = Totamount / Counter;
Eval(h) Yards = Sqyards **.5;
Eval(h) Pay = Hourlyrate * 40 + 1.5 * Hourlyrate * (Hoursworked - 40);
```

Improving Eval Precision

- *Resdecpos (Result Decimal Positions) rule ensures no decimal accuracy will be lost in complex expression
  - Not default
  - Default is *Maxdigits (Maximum Digits) rule, which may lose intermediate decimal positions
- Specify rule in Ctl-opt instruction at top of program
  - Or use (R) extender for a single expression

```
Ctl-opt Expropts(*Resdecpos);
```
// this is the recommended technique for improved accuracy

```
Eval(hr) Pay = Hourlyrate * 40 + 1.5 * Hourlyrate * (Hoursworked - 40);
```

Using Built-in Functions with Numeric Data

- Arithmetic functions
  - %Div (Integer Division)
  - %Rem (Integer Remainder)
  - %Abs (Absolute Value)
  - %Sqrt (Square Root)
- Data conversion functions
  - %Dec (Convert to Packed Decimal Format)
    - %Dech (Convert to Packed Decimal Format with Half-adjust)
  - %Int (Convert to Signed Integer Format)
    - %Inth (Convert to Signed Integer Format with Half-adjust)
  - %Uns (Convert to Unsigned Integer Format)
    - %Unsh (Convert to Unsigned Integer Format with Half-adjust)

%Div (Integer Division)

- %Div (Integer division) performs integer division of two numbers, returns integer quotient
  - Arguments must be numeric values, no decimals

```
// Convert total minutes to hours by dividing total minutes by 60.
// Hours is accurate to the number of decimal places defined for it,
so it could include digits behind the decimal
Hours = Minutes/60;

// The %Div function will return only the integer portion of Hours,
// regardless of how many decimal places are defined for it.
WholeHours = %Div(Minutes:60);
```

%Rem (Integer Remainder)

- %Rem (Remainder) returns integer remainder when dividing two numbers
- Not necessary to actually perform division to return remainder

```
// Convert total minutes to hours and minutes by dividing total
// minutes by 60 to get WholeHours and then returning the remainder as
// WholeMinutes.
WholeHours = %Div(Minutes:60);
WholeMinutes = %Rem(Minutes:60);
```

%Sqrt (Square Root)

- %Sqrt (Square root) returns square root of positive numeric expression
  - Accurate to scale of result

```
Eval(h) Hypotenuse = %Sqrt(Length**2 + Width**2);
```

%Dec (Convert to Packed Decimal Format)

- %Dec (Convert to Packed Decimal Format) converts result of numeric, character, or date expression to packed decimal format of specified precision and scale
  - %Dech (Convert to Packed Decimal Format with Half-adjust) rounds result

```
Result = %Dec(Hours*Rate : 7 : 2);

// This formula calculates a monthly payment, converting it to a
// 13 digit packed number with two decimal places
Payment = %Dec(Principal * (((Interest / 12) *
                (1 + (Interest / 12)) ** Months) /
                (((1 + (Interest / 12)) ** Months) - 1))
                : 13 : 2);
```

%Dec (Convert to Packed Decimal Format)

- %Dech (Convert to Packed Decimal Format with Half-adjust) rounds result
- Useful for rounding portion of expression

```
// This formula calculates total pay, by first calculating regular pay
// and overtime pay, rounding each component (accurate to 11 digits
// with two decimal places), then adding them together
Pay = %Dech(Hourlyrate * 40 : 11 :2) +
      %Dech(1.5 * Hourlyrate * (Hoursworked - 40) : 11 : 2);
```

Integer Conversion Functions

- %Int (Convert to Signed Integer Format) converts result of numeric, character, or date expression to signed integer
  - %Inth (Convert to Signed Integer Format with Half-adjust) rounds result
- %Uns (Convert to Unsigned Integer Format) converts to unsigned integer
  - %Unsh (Convert to Unsigned Integer Format with Half-adjust) rounds result

```
// Find the area of a circle as a rounded whole number
Area = %Inth(3.14159 * Radius**2);
```

Data Conversion Functions

- Conversion functions can convert character expressions to numeric representation
- Character value must consist of numeric information, 0-9
  - Can include sign (either preceding or following), decimal point, blanks

```
Dcl-s Var1   Char(11)    Inz('123.456789-');
Dcl-s Var2   Char(11)    Inz(' + 9 . 876 ');
Dcl-s Result Packed(9:5);

Result = %Dec(Var1:5:2);          // Result = -123.45000
Result = %Dech(Var2:5:2);         // Result =    9.88000
Result = %Dech(Var2:5:2) / 3;     // Result =    3.29333
Result = %Int(Var2:5:2);          // Result =    9.00000
```

Legacy Code: Fixed Format Operations
Glance through this section – will cover later in course

- Add
- Div (Divide)
- Mult (Multiply)
- Mvr (Move Remainder)
- Sqrt (Square Root)
- Sub (Subtract)
- Z-add (Zero And Add)
- Z-sub (Zero And Subtract)

Legacy Code: Fixed Format Operations

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C     Totalsales    Add       Orderamount   Totalsales
        // Same as Eval      Totalsales = Totalsales + Orderamount

C                   Add       Orderamount   Totalsales
        // Same as Eval      Totalsales += Orderamount

C                   Z-add     .0756         Taxrate
        // Same as Eval      Taxrate = .0756

C     UYear         Div       4             X
C                   Mvr                     Y
        // Same as Eval      X = %Div(Uyear:4)
        //          Eval      Y = %Rem(Uyear:4)

C                   Sqrt      X             Y
        // Same as Eval      Y = %Sqrt(X)
```

Legacy Code: Result Field Definition

- Fixed-format C-specs can define result fields
  - Not recommended
- Positions 64-68 indicate length in bytes or digits
- Positions 69-70 indicate decimals for numeric fields
  - Blanks indicate character field

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C                   Z-add     .0756         Taxrate           7 7
                                          // Taxrate is Packed(7:7)
C                   Move      'Y'           Taxable           1
                                          // Taxable is Char(1)
```

Legacy Code: Resulting Indicators

- Resulting indicators reflect result of numeric operation
  - Position 71-72 (Hi) is set on if result is positive
  - Position 73-74 (Lo) is set on if result is negative
  - Position 75-76 (Eq) is set on if result is zero

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C     UYear         Div       4             X
C                   Mvr                     Y                        50

C     Taxable       Mult      Taxrate       Tax                   5151
```